Rigid body simulator

1 Introduction

This document will describe a very simple rigid body simulator we will use to design and develop our drone control system. Too efficiently design the system we will need simulations. In this module we will design a very simple simulator which will solve Newton's equations. In doing so, we can predict under certain approximations how the drone will behave when we control it.

2 Reference system

Same explanation as in the AHRS system (body and world frame of reference). We'll need to make a reference later.

3 Rigid body dynamics

In this section, we discuss the rigid-body dynamics required to solve the equations of motion. Throughout this chapter, all vectors are expressed in the non-inertial world reference frame, denoted by n.

The objective is to determine the system state, which is defined by the state vector

$$\mathbf{x} = \begin{bmatrix} \mathbf{r} \\ \mathbf{v} \\ \mathbf{p} \\ \boldsymbol{\omega} \end{bmatrix},$$

where

- $\mathbf{r}$ is the position of the body's center of mass in the world frame,
- $\mathbf{v}$ is the linear velocity of the center of mass,
- $\mathbf{p}$ is the orientation of the body, represented as a quaternion,
- $\boldsymbol{\omega}$ is the angular velocity of the body, expressed in the world frame.

At the start of the simulation, the state vector is initialized to a prescribed initial condition, $\mathbf{x}_0$, chosen to represent the scenario under consideration.

The evolution of the system is governed by the equations of motion, which define the time derivative of the state vector,

$$\dot{\mathbf{x}} = \begin{bmatrix} \mathbf{v} \\ \mathbf{a} \\ \dot{\mathbf{p}} \\ \boldsymbol{\alpha} \end{bmatrix}$$

- $\mathbf{v}$ is the linear velocity of the center of mass,
- $\mathbf{a}$ is the acceleration of the body
- $\dot{\mathbf{p}}$ is the orientation rate of change of the body,
- $\boldsymbol{\alpha}$ is the angular acceleration of the body, expressed in the world frame.

At each simulation step, the state derivative $\dot{\mathbf{x}}$ is computed from the current state $\mathbf{x}$ together with the applied external forces and moments. A numerical integration scheme is then used to advance the state from the current time step to the next. This procedure is repeated iteratively until the desired simulation time has been reached.

In the next sections we will discuss how to obtain each element of the state derivative vector. After that we discuss two integration schemes which can be used too obtain the next state.

3.1 Computing linear acceleration

We start by computing the total force $\mathbf{F}$ (described in the n frame) by summing all the known external forces acting on the body:

$$\mathbf{F} = \sum_i \mathbf{F}_i \tag{1}$$

Now we can compute the acceleration the rigid body experiences:

$$\mathbf{a} = \frac{\mathbf{F}}{m}$$

In this case m is the total mass of the rigid body.

3.2 Computing angular acceleration

To determine the angular acceleration, we first compute the net torque $\boldsymbol{\tau}$ acting on the rigid body about its center of mass. This is achieved by summing all pure torques alongside the moments generated by forces applied at a distance from the center of mass:

$$\boldsymbol{\tau} = \sum_i \boldsymbol{\tau}_i + \sum_j (\mathbf{r}_j \times \mathbf{F}_j) \tag{2}$$

where $\mathbf{r}_j$ represents the position vector from the body's center of mass to the point of application of the force $\mathbf{F}_j$, and $\times$ denotes the vector cross product.

Because the equations of motion are expressed in the world reference frame n, the constant body-frame inertia tensor $\mathbf{I}_{\text{body}}$ must be transformed into the world frame at each time step. Using the rotation matrix $\mathbf{R}$ derived from the current orientation quaternion $\mathbf{p}$, the world inertia tensor $\mathbf{I}$ is obtained via the following change of basis:

$$\mathbf{I} = \mathbf{R}\mathbf{I}_{\text{body}}\mathbf{R}^T \tag{3}$$

where $\mathbf{R}^T$ is the transpose of the rotation matrix.

The angular acceleration $\boldsymbol{\alpha}$ can then be derived using Euler's equations of motion in the world frame, accounting for both the net torque and gyroscopic effects:

$$\boldsymbol{\alpha} = \mathbf{I}^{-1}\left(\boldsymbol{\tau} - \boldsymbol{\omega} \times (\mathbf{I}\boldsymbol{\omega})\right) \tag{4}$$

where $\boldsymbol{\omega}$ is the current angular velocity in the world frame.

3.3 Computing orientation derivative

The rate of change of the orientation quaternion, $\dot{\mathbf{p}}$, is determined directly from the current angular velocity $\boldsymbol{\omega}$ and the current orientation $\mathbf{p}$. Because the angular velocity vector is expressed in the world frame n, the quaternion derivative is given by:

$$\dot{\mathbf{p}} = \frac{1}{2}\boldsymbol{\omega}_q \otimes \mathbf{p} \tag{5}$$

where $\otimes$ denotes the quaternion multiplication operator, and $\boldsymbol{\omega}_q$ is the angular velocity vector promoted to a pure quaternion with a zero scalar component:

$$\boldsymbol{\omega}_q = \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix} \tag{6}$$

In practice, this quaternion multiplication can be written as a matrix-vector product to make numerical implementation straightforward:

$$\dot{\mathbf{p}} = \frac{1}{2}\begin{bmatrix} -\boldsymbol{\omega} \cdot \mathbf{p}_{1:3} \\ p_0\boldsymbol{\omega} + \boldsymbol{\omega} \times \mathbf{p}_{1:3} \end{bmatrix} \tag{7}$$

where p_0 is the scalar component of the quaternion $\mathbf{p}$, and $\mathbf{p}_{1:3}$ represents its vector component.

Because numerical integration schemes do not naturally preserve the unit length of a quaternion, the orientation quaternion $\mathbf{p}$ must be re-normalized to unit length ($\|\mathbf{p}\| = 1$) at the end of every simulation step to prevent numerical drift from distorting the rotation.

3.4 Linear and angular velocities

By examining the state vector $\mathbf{x}$ and its time derivative $\dot{\mathbf{x}}$, it becomes apparent that determining the linear and angular velocities for the state derivative equation $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ is straightforward. Rather than requiring dynamic computation from external forces or torques, these velocity terms are already explicitly stored as components within the state vector itself.

4 Integration Schemes

In this section, we describe the two integration schemes implemented in our simulator: the Forward Euler method and the fourth-order Runge-Kutta (RK4) method.

4.1 Forward Euler

The Forward Euler integration scheme is the simplest explicit method for solving ordinary differential equations. Given the current state $\mathbf{x}_n$ and its time derivative $\dot{\mathbf{x}}_n = f(t_n, \mathbf{x}_n)$, the next state $\mathbf{x}_{n+1}$ is computed using the following equation:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t \cdot \dot{\mathbf{x}}_n \tag{8}$$

While computationally lightweight and straightforward to implement, this first-order method is highly susceptible to accumulation errors and numerical instability. It assumes that the state derivative remains constant over the entire time interval Δt, which rarely holds true in dynamic systems.

4.2 Fourth-Order Runge-Kutta (RK4)

To achieve higher accuracy and stability, the simulator also implements the fourth-order Runge-Kutta (RK4) method. Rather than relying solely on the derivative at the beginning of the time step, RK4 samples the derivative $f(t, \mathbf{x})$ at four different points across the interval to compute a weighted average slope.

The update step is defined as follows:

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \tag{9}$$

where the intermediate slopes $\mathbf{k}_i$ are evaluated sequentially:

$$\mathbf{k}_1 = f(t_n, \mathbf{x}_n) \tag{10}$$

$$\mathbf{k}_2 = f\left(t_n + \frac{\Delta t}{2}, \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_1\right) \tag{11}$$

$$\mathbf{k}_3 = f\left(t_n + \frac{\Delta t}{2}, \mathbf{x}_n + \frac{\Delta t}{2}\mathbf{k}_2\right) \tag{12}$$

$$\mathbf{k}_4 = f(t_n + \Delta t, \mathbf{x}_n + \Delta t\mathbf{k}_3) \tag{13}$$

RK4 systematically samples the slope four separate times across the time step. By doing this clever averaging the simulation ends up being more accurate and stable.

Naturally, this increased accuracy comes at a higher computational cost. While Forward Euler requires only a single derivative evaluation per time step, RK4 requires four, making each step four times more expensive to compute.